

Project Report - Transaction network analysis

Ilija Chrchev

89241041@student.upr.si

University of Primorska, FAMNIT

Koper, Slovenia

ABSTRACT

This report presents a system for detecting wash trading in Ethereum NFT markets through linkability network analysis. Wash trading is a form of market manipulation in which traders artificially inflate the volume or price of an asset by trading it back and forth between accounts they secretly control. To uncover such behavior, we build a linkability network by performing a bounded breadth first search (BFS) on the Ethereum Transaction Network and connecting any two traders that share a path of transactions within a fixed depth.

The same algorithm is implemented in three execution modes, a **sequential** version, a **multithreaded parallel** version and a **distributed** version based on MPI. The report describes the design of each version, the classes and methods I am using, as well as the problems I encountered along the way of building, together with how each problem was approached and solved.

1 INTRODUCTION

Non-fungible tokens known as NFTs are digital assets traded on public blockchains like Ethereum. Because every transaction is recorded and visible, these markets look transparent, but this transparency is often abused through *wash trading*, where the same person controls both sides of a trade to inflate volume and prices.

Spotting wash trading is hard because Ethereum addresses are anonymous and one person can own many wallets. What we can see is how money moves between them. If two NFT traders are connected by a short chain of transactions, they are likely linked. This is the idea behind *linkability network analysis*.

2 PROBLEM STATEMENT

Given the Ethereum Transaction Network and the set of traders extracted from the Bored Ape Yacht Club (BAYC) collection, the goal is to find all pairs of traders that are connected by a path of transactions of length at most k , while ignoring paths that pass through blacklisted addresses such as exchanges and other high traffic services that almost everyone interacts with.

The output is a linkability network, a graph where each node is a BAYC trader and each edge marks a pair of traders that are linkable within depth k . The challenge is doing this efficiently on a dataset of around 5 GB, which is what motivates the three execution modes presented in the next sections.

3 PROJECT STRUCTURE

The project is organised into three packages plus the three entry points, one per execution mode. The classes in `model`, `utils` and the first two classes of `builder` are shared by all three modes, while each mode has its own linkability builder.

`src/`

`SequentialMain.java`

`ParallelMain.java`

`DistributedMain.java`

`model/`

`AddressMapper.java`

`Graph.java`

`builder/`

`GraphBuilder.java`

`NFTTraderLoader.java`

`LinkabilityBuilder.java`

`ParallelLinkabilityBuilder.java`

`DistributedLinkabilityBuilder.java`

`utils/`

`BlacklistReader.java`

`Logger.java`

`data/`

`prog3ETNsample.csv`

`boredapeyachtclub.csv`

`blacklist/`

`batch.json`

`bridges.json`

`cex.json`

`defi.json`

`dex.json`

`tornado.json`

`model` holds the data structures, `builder` holds everything that reads input or runs the BFS, and `utils` holds the two helpers. Each `Main` class only wires the stages together and reports timings, so the three modes differ from each other in exactly one class.

4 SHARED CLASSES

Before describing the three implementations, it is worth presenting the classes that all of them rely on. These classes handle loading the data, representing the graph and providing the helpers used by the BFS, regardless of execution mode. They live in the `model`, `builder` and `utils` packages.

4.1 AddressMapper

Ethereum addresses are 42 character hexadecimal strings, which are expensive to store and compare. `AddressMapper` assigns each unique address an integer ID the first time it is seen, so the rest of the system can work with primitive `int` values instead of strings. This single change drastically reduces memory usage and speeds up every lookup in the graph.

Only the forward mapping from address to ID is kept. An earlier version also stored the reverse mapping from ID back to address, but nothing ever needed it, since the output CSV is written with integer IDs. Removing it saved a five million entry map.

Main methods:

- `getOrCreateId(String address)` returns the ID of an address, creating a new one if it has not been seen before.
- `size()` returns the total number of mapped addresses, which is also the number of nodes in the graph.
- `clear()` releases the internal map once it is no longer needed, to free memory before the BFS starts.

4.2 Graph

Graph stores the Ethereum Transaction Network as a map from each node to its list of neighbours, where the neighbour list is an `ArrayList<Integer>`. The BFS only ever needs to iterate over a node's neighbours, so a plain list is enough and keeps the class small.

Main methods:

- `addEdge(int from, int to)` adds a directed edge between two nodes.
- `getConnected(int nodeId)` returns the list of neighbours of a node, or an empty list if the node has none.
- `removeDuplicateEdges()` removes duplicate entries from each neighbour list, which come from repeated transactions between the same two addresses.
- `edgeCount()` returns the number of edges in the graph.

4.3 GraphBuilder

GraphBuilder reads the ETN CSV file line by line and builds the Graph together with the AddressMapper. For each transaction it extracts the from and to addresses, skips the row if either of them is blacklisted, maps the addresses to integer IDs and adds the edge to the graph. Self loops are skipped.

Main methods:

- `buildFromETN(String etnFilePath)` performs the full read of the CSV file and populates the graph.
- `isBlackListed(String address)` checks whether a given address is in the blacklist.
- `getGraph()` and `getAddressMapper()` return the populated structures so they can be used by the next stages.

4.4 NFTTraderLoader

NFTTraderLoader reads the BAYC CSV file and produces the set of trader IDs that the BFS will run from. For each transfer it considers both the sender and the receiver, skipping the null address, which represents minting and burning, and any blacklisted address. The remaining addresses are mapped to integer IDs through the same AddressMapper used for the graph, so traders and graph nodes share the same ID space.

Main methods:

- `loadTraders(String nftFilePath)` reads the file and returns the set of trader IDs.
- `isBlacklisted(String address)` same blacklist check as in GraphBuilder.

4.5 BlacklistReader

BlacklistReader loads all blacklist JSON files from a folder and merges them into a single `Set<String>`. The blacklist contains hub addresses that should be ignored during graph building and trader loading, since they would otherwise produce false connections between unrelated traders.

Main method:

- `loadBlacklist(String folderPath)` reads every `.json` file in the folder, extracts every string starting with `0x` and returns the merged set.

4.6 Logger

Logger is a small utility that prints timestamped log messages to the console at different levels: info, warn, error, debug and success. Each level is printed in a different color so that progress, warnings and errors are easy to distinguish during long runs.

5 SEQUENTIAL IMPLEMENTATION

The sequential implementation is the baseline version of the system. It runs the entire pipeline on a single thread: load the blacklist, build the graph, load the traders, then run BFS from each trader one after another. The entry point is `SequentialMain`, and the BFS itself is encapsulated in the `LinkabilityBuilder` class.

5.1 SequentialMain

SequentialMain ties all the shared classes together and runs them in order. It reads the command line arguments (max depth, ETN file, NFT file, blacklist folder, output file) or falls back to default values, then executes the pipeline:

- (1) Load the blacklist with `BlacklistReader`.
- (2) Build the ETN graph with `GraphBuilder`, then call `removeDuplicateEdges()` on the graph to remove repeated neighbours.
- (3) Load the BAYC traders with `NFTTraderLoader`.
- (4) Run `LinkabilityBuilder.buildLinkabilityNetwork()` to perform the BFS and write the output CSV.

Each stage is timed individually and the total runtime is reported at the end.

5.2 LinkabilityBuilder

LinkabilityBuilder is the heart of the sequential version. It holds a reference to the graph, the set of trader IDs, the maximum depth and the node count, and exposes a single public method, `buildLinkabilityNetwork(String outputFile)`, which runs a bounded BFS from every trader and writes one CSV line for each linkable pair found.

The method works as follows. First, the trader set is converted into a boolean lookup array `isTrader[]` of size `nodeCount`, so checking whether a node is a trader becomes a single array access during the BFS. Three primitive arrays are allocated once and reused across all BFS runs: `seen[]` for visited markers, `distance[]` for BFS levels, and `queue[]` as the BFS frontier.

The queue is allocated once with size `nodeCount`. This is always enough, because each node is added to the queue at most once per BFS run, so no resizing logic is needed.

For each source trader, a fresh BFS is started by incrementing a counter `bfsId` and using it as the visited marker. A node counts as visited in the current BFS only if `seen[node] == bfsId`. This avoids clearing a five million entry array between every one of the 19,714 traversals.

During the traversal, when a neighbour is reached at distance `n`, the BFS checks whether that neighbour is itself a trader. If it is and it is not the source, a line `source,neighbour,n` is written to the output file and the link counters are updated.

Main method:

- `buildLinkabilityNetwork(String outputFile)` runs the full BFS over all traders and writes the resulting linkability network.

5.3 Challenges Faced

A few problems came up during the development of the sequential version.

The first was raw scale. The ETN CSV is around 5 GB and contains millions of lines, so any approach that allocates per row objects or builds intermediate structures runs out of memory before it finishes. The fix was to read the file with a `BufferedReader` line by line and to push everything into the integer graph immediately, without keeping the original strings around.

The second was duplicate edges. The same pair of addresses can transact many times, which means the same edge gets added repeatedly to the adjacency list. This made the BFS slower without changing its result. The `removeDuplicateEdges()` method on `Graph` solves this by removing repeated entries from each neighbour list. On this dataset it cuts the edge count from 10,256,630 to 5,943,986.

The third was a subtle bug related to sizing the BFS arrays. The first version used `graph.nodeCount()` to size `seen[]` and `distance[]`, but that only counts nodes that have outgoing edges. Some addresses appear only as edge targets, never as sources, and they have integer IDs that fall outside that range. This caused an `ArrayIndexOutOfBoundsException` when the BFS reached such a node. The fix was to size all BFS arrays based on `addressMapper.size()` instead, which counts every mapped address.

6 PARALLEL IMPLEMENTATION

The parallel implementation keeps the same algorithm as the sequential one but runs the BFS for many traders at the same time. Since each BFS starts from a different source and only reads from the graph, the traders can be processed independently as long as the workers do not interfere with each other through shared state. The entry point is `ParallelMain`, and the BFS logic lives in `ParallelLinkabilityBuilder`.

6.1 ParallelMain

`ParallelMain` performs the same pipeline as `SequentialMain`: load the blacklist, build the graph, remove duplicate edges, load the traders. The only difference is the last step, where it constructs a `ParallelLinkabilityBuilder` instead of a `LinkabilityBuilder`. It also logs the number of available processors at the start so the run is reproducible.

6.2 ParallelLinkabilityBuilder

`ParallelLinkabilityBuilder` runs the BFS in parallel by starting one worker thread per available processor and splitting the traders between them. Each trader is handled by exactly one thread. To make this safe, three concerns are handled separately: per thread state, per thread counters and concurrent file output.

Splitting the work. The traders are copied into an `int[]` so they can be split by index. Worker `t` handles the traders at indices `t, t + threadCount, t + 2*threadCount` and so on. This is the same interleaved split used in the distributed version, and it balances the work well, since BFS cost varies a lot from one trader to the next and this spreads heavy and light traders evenly across the threads.

Per thread state. Each BFS needs three big arrays: `seen[]`, `distance[]` and `queue[]`. Each worker allocates its own set once at the start of its `run()` method and reuses them for every trader it handles. Because the arrays are local to the thread, no two threads can ever touch the same BFS state, so no synchronization is needed for them. Each worker also keeps its own `bfsId` counter, so the stamping trick from the sequential version works per thread.

Per thread counters. Each worker counts its own links and its own per weight breakdown into a private `long[]`. Since these are never shared while the threads run, they need no locking. Once all workers have finished, the main thread adds the per worker counts together into the final totals.

Concurrent file output. The one thing the workers do share is the output file. All of them write to a single `BufferedWriter`, and every write goes inside a synchronized (writer) block, so only one thread is inside the writer at any moment. The lock is held for a single short line, so threads spend almost all of their time doing BFS work outside it.

Main method and inner class:

- `buildLinkabilityNetwork(String outputFile)` starts the worker threads, waits for all of them with `join()`, then sums their counts and closes the writer.
- `BfsWorker` is a private inner class that extends `Thread`. It holds the trader array, the lookup table and the shared writer, and its `run()` method does the BFS for that worker's share of the traders.

6.3 Challenges Faced

A challenge was safe file output. Writing to the shared writer from every worker without a lock corrupts the file, so every write goes inside a synchronized block on the writer object.

7 DISTRIBUTED IMPLEMENTATION

The distributed implementation runs the same BFS across multiple processes using MPI. Rank 0 loads the data and sends it to the others, then every rank handles its own share of the traders. The entry point is `DistributedMain`, and the BFS logic lives in `DistributedLinkabilityBuilder`. The project uses MPJ Express 0.44 in multicore mode.

7.1 DistributedMain

DistributedMain runs on every rank in parallel. It starts with `MPI.Init(args)` and reads the rank and total size from `MPI.COMM_WORLD`. Only rank 0 loads the blacklist, builds the graph, removes duplicate edges and loads the traders. It then sends the graph and the trader set to the other ranks through `MPI.COMM_WORLD.Bcast()`. This is still a **replicated graph** design, since every rank ends up with the full ETN graph in memory, but the file is read only once instead of once per rank.

To send the graph, rank 0 flattens the adjacency lists into two `int[]` arrays: `offsets`, which holds the start index of each node's neighbours, and `neighbours`, which holds all neighbours in node order. The neighbours of node i are the entries between `offsets[i]` and `offsets[i+1]`. Three sizes are broadcast first so the other ranks know how big to allocate their arrays, then the two arrays and the trader array follow. All four broadcasts happen in the same order on every rank, otherwise the ranks would deadlock waiting on different messages.

The trader set is converted to a sorted `int[]` before being broadcast, so that every rank sees the same order. This matters because the BFS work is split by index, and if the order differed between ranks, two ranks could end up working on the same trader or skipping one.

Before the BFS starts, all ranks synchronize with `MPI.COMM_WORLD.Barrier()` and timing starts from `MPI.Wtime()`. Each rank constructs its own `DistributedLinkabilityBuilder` and calls `buildLocalPart()` to write its share of the output to a temporary file named `output_distributed.csv.rankN.tmp`. After another barrier, the local link counts are combined with `MPI.Reduce(..., MPI.SUM, 0)` so rank 0 ends up with the global totals.

Finally, rank 0 merges the temporary files into a single output CSV by concatenating them in rank order, then logs the totals and the runtime. `MPI.Finalize()` ends the program.

7.2 DistributedLinkabilityBuilder

`DistributedLinkabilityBuilder` runs the same BFS as the sequential version, with three differences: it reads the graph from the two flattened `int[]` arrays instead of a `Graph` object, it works on a slice of the trader array instead of the full set, and it writes to a rank specific temporary file.

The slice is selected with interleaved partitioning:

```
for (int index = rank;
    index < allTraders.length;
    index += size) {
    int source = allTraders[index];
    ...
}
```

With 4 ranks, rank 0 processes traders at indices 0, 4, 8, 12 and so on, rank 1 takes 1, 5, 9, 13, and so on. This is simpler than splitting the array into contiguous blocks and gives a more balanced workload, since BFS cost varies a lot from one trader to the next and contiguous blocks could land mostly heavy or mostly light traders on the same rank.

Each rank has its own `seen[]`, `distance[]` and `queue[]` arrays and uses the same BFS stamping. The output goes to a per rank temporary file rather than a shared one, so there is no need for

locks between ranks. Only rank 0 writes the CSV header at the start of its temporary file, so when the files are concatenated the header appears exactly once.

Main method:

- `buildLocalPart(String tempFile)` runs the BFS over the rank's share of traders and returns its local link by weight counts.

7.3 Approach

The BFS only reads from the graph, so the traders can be split across ranks without any communication during the BFS itself. The only real work is dividing the traders and putting the output back together at the end.

A replicated graph was used instead of a partitioned one because the dataset fits in memory. Splitting the graph between ranks would force them to send messages every time a BFS crossed a boundary, which would be much slower than giving every rank its own copy.

Each rank writes its results to its own temporary file and rank 0 merges them at the end, so no rank ever waits for another while writing.

7.4 Challenges Faced

The first challenge was memory. In multicore mode every rank is a thread inside a single JVM, so all ranks share one heap. The first version had every rank open and parse the 5 GB file independently, which meant the graph was held in memory as many times as there were ranks. It ran out of memory at higher rank counts. The fix was to have only rank 0 parse the file and broadcast the flattened graph, and to clear the address mapper and the blacklist once the trader set is built, before the BFS starts.

The second was making the split deterministic. The trader set is a `Set<Integer>`, which has no fixed order, so two ranks could disagree on which trader is at index 5. Converting the set to a sorted `int[]` before broadcasting fixed this.

8 EXPERIMENTAL SETUP

All experiments were run on a single machine with an AMD Ryzen 7 7840HS (8 cores, 16 threads) and 16 GB of RAM, running Java 21. The dataset is the Ethereum Transaction Network CSV at around 5 GB. After blacklist filtering and duplicate removal the graph has 5,016,686 nodes and 5,943,986 edges. The trader set comes from the Bored Ape Yacht Club transfers and contains 19,714 unique addresses.

All measurements were taken at depth 7. The parallel version uses all 16 hardware threads automatically through `Runtime.getRuntime().availableProcessors()`. The distributed version uses MPJ Express 0.44 in multicore mode, where each rank is a separate thread inside one JVM process on the same machine. Four rank counts were tested: `-np 2`, `-np 4`, `-np 6` and `-np 8`.

All six configurations produced exactly 988,249 links with an identical breakdown per weight, which confirms that the parallel and distributed versions are correct.

9 RESULTS

9.1 Runtime Comparison

Table 1 reports two numbers for every configuration. The total runtime covers the whole program, including reading the 5 GB CSV, building the graph, removing duplicate edges and loading the traders. The BFS column covers only the traversal phase, which is the part that is actually parallelised.

Table 1: Runtime at depth 7, total and BFS phase only

Mode	Total	BFS only
Sequential	228.8 s	185.0 s
Parallel (16 threads)	66.2 s	23.0 s
Distributed (-np 2)	104.1 s	57.8 s
Distributed (-np 4)	69.3 s	30.2 s
Distributed (-np 6)	72.2 s	24.9 s
Distributed (-np 8)	67.7 s	21.1 s

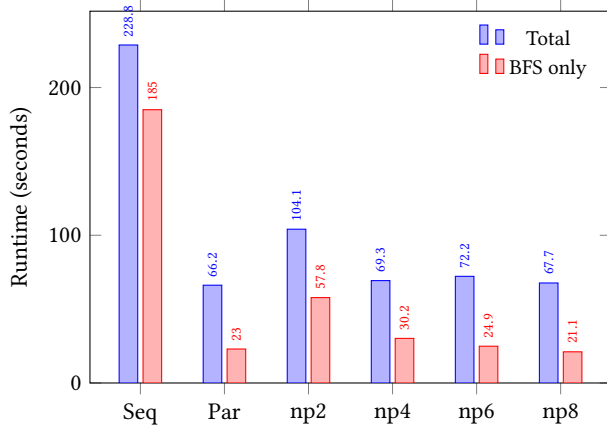


Figure 1: Total runtime and BFS phase at depth 7.

10 CONCLUSION

This report presented a system for detecting wash trading in Ethereum NFT markets through linkability network analysis. The core algorithm is a bounded BFS from each BAYC trader over the full Ethereum Transaction Network, with hub addresses filtered out by a blacklist. The same algorithm was implemented in three modes: sequential, multithreaded parallel, and distributed using MPI.

The sequential version was tuned to be fast enough to serve as a real baseline, through integer address mapping, BFS stamping and a precomputed trader lookup table. The parallel version started one worker thread per processor, split the traders between them by interleaved index, gave each thread its own BFS arrays and counters, and shared only the output writer, guarded by a lock. The distributed version replicated the graph across ranks by flattening it into two integer arrays and broadcasting them, split the traders with interleaved partitioning, and wrote one temporary file per rank that rank 0 merged at the end.

All six configurations produced exactly the same 988,249 links. The parallel mode reached a $8.0\times$ BFS speedup over the sequential version, and the distributed mode a $2.74\times$ BFS speedup going from 2 to 8 ranks, with the total runtime in both cases bounded by the serial reading of the 5 GB input file.